

Saliency-Aware Deep Residual Networks for Plant Phenotype Prediction

Jake Goode, Madhurika Madhu, Raeein Bagheri, and Yan Yan*

School of Computer Science

University of Guelph, Guelph, Canada

{goode, mmadhu, bagherir, yyan15}@uoguelph.ca

Abstract—Predicting complex phenotypes from genomic data is challenging in biology and machine learning. We propose a deep learning approach using a saliency-aware residual neural network (ResNet) for such prediction and applied to plant single-nucleotide polymorphism (SNP) data. Our model encodes genome-wide SNPs in a one-hot tensor and employs a 1D ResNet architecture with a custom inverse square root unit (ISRU) activation to stabilize regression outputs. We train the network with 10-fold cross-validation on both imputed and non-imputed (QA) genotype datasets. Experimental results on a soybean dataset show that our model achieves a mean Pearson correlation coefficient (PCC) between 0.39 and 0.64 for different soybean phenotypes. To interpret the model, we compute gradient-based saliency maps that highlight influential SNPs. The saliency analysis reveals a sparse set of top-ranking SNPs with outsized impact on the phenotype prediction, aligning with known genomic markers. We provide a fully containerized (Docker) pipeline for reproducibility. This work demonstrates that deep residual networks can yield accurate phenotype predictions while also identifying candidate genetic variants, bridging the gap between predictive accuracy and interpretability in genome-wide association studies. The model is available at <https://github.com/ProductiveOwl/Soybean-CNN>

Index Terms—Deep learning, Genome-wide association study (GWAS), Saliency mapping

I. INTRODUCTION

With the availability of whole genome sequencing and large scale phenotyping, studies on whole genome population data and the phenotypes are emerging. Among them, genome-wide association studies (GWAS) are popular for investigating the relationships between genetic variants and traits of interest [1]–[3]. The most common variants used are single-nucleotide polymorphisms (SNPs), which are the single nuclear base-pair changes in genomes. By scanning SNPs on the entire genome, GWAS performs statistical analysis to reveal significant SNPs associated with traits, and such studies have uncovered valuable information on the genetic architecture of complex traits [1]. The output from GWAS typically includes a list of SNPs along with their p-values, indicating the associations with the phenotype.

Despite its popularity, GWAS also faces challenges. Different programs may require a specific installation and input data format. GWAS models may miss important SNPs associated with traits with a certain cut-off value p [3]. Different GWAS programs often produce dissimilar association results with no or few shared significant SNPs [4].

Phenotype prediction from genetic data has important applications in genomics, agriculture, and medicine. A phenotype (observable trait) such as plant height is often influenced by many genetic variants (e.g. SNPs). Traditional approaches like genomic best-linear unbiased prediction (GBLUP) and related Bayesian models have been widely used to predict quantitative traits from SNP profiles [5]. While effective with sufficient data, these linear models assume additive effects and struggle to capture complex interactions (epistasis) among SNPs [5]. Moreover, missing genotype values are common and typically require imputation, which can introduce bias [5].

Deep learning offers a powerful alternative that can model nonlinear relationships and interactions in high-dimensional genomic data. In particular, convolutional neural networks (CNNs) have recently been applied to genome-wide data by treating SNP vectors as “images” or sequences that convolutional filters can scan for predictive patterns [5]. Several studies have demonstrated the promise of CNNs in genome-wide association studies (GWAS). CNNs in GWAS and Genomic Prediction had early efforts to apply deep learning to genomic selection and GWAS demonstrated that CNNs can automatically capture linkage patterns and SNP interactions. Liu et al. (2019) [5] developed a dual-stream 1D CNN to predict soybean traits: one stream had successive convolution filters of size 4 and 20, the other a single filter of size 4, with a skip connection merging them. This residual CNN improved prediction accuracy for soybean yield and other traits over linear models and identified significant SNP markers that corresponded to known quantitative trait loci (QTLs). Chen et al. (2021) [6] transformed selected SNP data into pixelated images (so-called artificial image objects) and applied a CNN to classify schizophrenia risk, achieving up to 80% classification accuracy using around 44k SNPs as input features, illustrating that providing the network with more genomic data improved performance.

CNNs can successfully predict traits from SNPs and identify important markers, but interpretability remains a challenge. Integrating Residual neural networks (ResNet) could be a potential. ResNets were first introduced by He et al. [7] (2016) to enable very deep networks to train effectively by using skip connections that bypass one or more layers [5]. The skip connection adds the input of a layer to its output, helping gradients propagate and mitigating the vanishing gradient problem [5].

ResNets have since become a standard architecture in image recognition and have been applied successfully to sequential genomic data as well [5].

In this study, we focus on predicting phenotypes from plant genome-wide SNP data by adopting a ResNet-style skip connection to improve learning stability on genomic sequences. Our work differentiates itself by tightly integrating a ResNet CNN with an interpretation mechanism (saliency analysis – a new saliency-aware deep residual network for SNP-based phenotype prediction), and by demonstrating a practical, reproducible pipeline on a real-world plant height prediction task.

Our contributions are threefold: (1) We design a lightweight 1D CNN with residual connections (ResNet) and a smooth activation function to predict a quantitative trait from high-dimensional SNP inputs. (2) We integrate gradient-based saliency mapping to highlight the most influential SNP loci for the trait, providing an explanation for the network’s predictions in terms of genetic features. (3) We implement the entire pipeline in a modular, memory-efficient manner and containerize it with Docker for reproducibility. By combining a ResNet architecture with saliency analysis, our approach produces accurate predictions while retaining the ability to pinpoint candidate genomic regions, thus bridging prediction and interpretation in deep learning-driven GWAS.

II. METHODOLOGY

A. Data Format and Preprocessing

The input data is needed in tab-delimited text files (one for imputed and one for non-imputed data, more on imputation in the following). Each row contains: a fold index (e.g. 1–10 for cross-validation grouping), the phenotype value, and the genotype of each SNP. Before passing the data into the CNN, we pre-process the fold numbers based on the user’s selection and complete the one-hot conversion before runtime.

If the dataset has missing values for SNPs, imputation is performed. In the imputed dataset, missing genotype calls have been filled in (imputed) using reference alleles, whereas in the non-imputed dataset, missing values are left as undefined. The imputed soybean dataset was created “using the MaCH software (Li, et al., 2010) [8] based on the HMM Approach” from Liu et al. (2019) [5] which uses the reference genome. Xavier et al. (2016) [9] found that this method had the best accuracy and phenotype predictability. Our work uses the imputed and non-imputed data provided in the GitHub repository by Liu et al. (2019) [5].

The model encodes SNP data using one-hot encoding assuming the SNPs are bi-allelic. For example, following the scheme by Liu et al. [5], a homozygous reference genotype (AA) could be [0,0,1,0], a heterozygous genotype (Aa) [0,1,0,0], a homozygous alternate (aa) [0,0,0,1], and a missing genotype [1,0,0,0]. This encoding yields a matrix of shape $N \times 4$ for N SNP markers for each individual. The model currently supports numerical genotypes that are encoded as 0, 1, or 2 (representing, e.g., the count of reference alleles) as this is the format of our experimental data. We understand that genotype

data could have varied formats (vcf, ped, etc.), and a separate tool for genotype format conversion is under development for the preprocessing of this model.

The model applies N -fold cross-validation (where $3 \leq N \leq 10$ for the current implementation). Therefore, each sample is assigned a fold number during preprocessing. We fix the seed number so that the fold is fixed for a certain sample for replication experiments.

We wrote a data loader to parse these files using pandas. The loader converts SNP columns to numeric dtype and phenotype to float, and stores the SNP matrix in a compact form. Importantly, instead of immediately expanding all SNPs into one-hot encoding in memory (which would be an array of shape [samples $\times$ SNPs $\times$ 4], potentially very large), we defer the one-hot conversion until runtime using a generator (described below). We represent the genotype matrix initially as a 2D array of type int8 (saving memory vs. float32) with shape (samples $\times$ SNPs), where values are in 0,1,2 (and possibly -1 or a sentinel for missing). This significantly reduces memory usage and allows us to handle thousands of SNP features without overflow.

B. Model Architecture

Our neural network model is a 1D convolutional ResNet designed to capture local sequence patterns of SNPs while accounting for long-range effects via a residual connection. Figure 1 illustrates the architecture. The input to the network is an $N \times 4$ one-hot matrix of N SNPs (sequentially ordered by genomic position). The network begins with two convolutional layers in sequence:

- **Conv1D Layer 1:** 10 filters, kernel size = 4, “same” padding. This layer scans through the SNP sequence with a window of 4 SNPs at a time, akin to looking at short haplotype patterns of length 4. It uses a linear activation (no non-linear function here) and includes L2 kernel regularization (penalizing weights) to prevent overfitting. The output is a feature map of shape $N \times 10$.
- **Conv1D Layer 2:** 10 filters, kernel size = 20, same padding, also linear activation. This layer takes the output of the first conv and looks at a wider window of 20 SNPs at a time, producing another $N \times 10$ feature map. By stacking a 4-length and a 20-length convolution, the network can detect both short-range and somewhat longer-range patterns in SNP sequences. These two conv layers in series constitute a stacked CNN stream similar to Liu et al.’s design [5].
- **Dropout:** After the two conv layers, we apply a dropout step. The default value is 75% drop rate. This is quite high, but the dataset is relatively small, and we want to aggressively prevent overfitting. This randomly zeros out some of the intermediate features, forcing the network to rely on multiple features rather than any single one. Due to the randomness of the dropout rate, this can cause slight variations in the final PCC values over multiple runs.

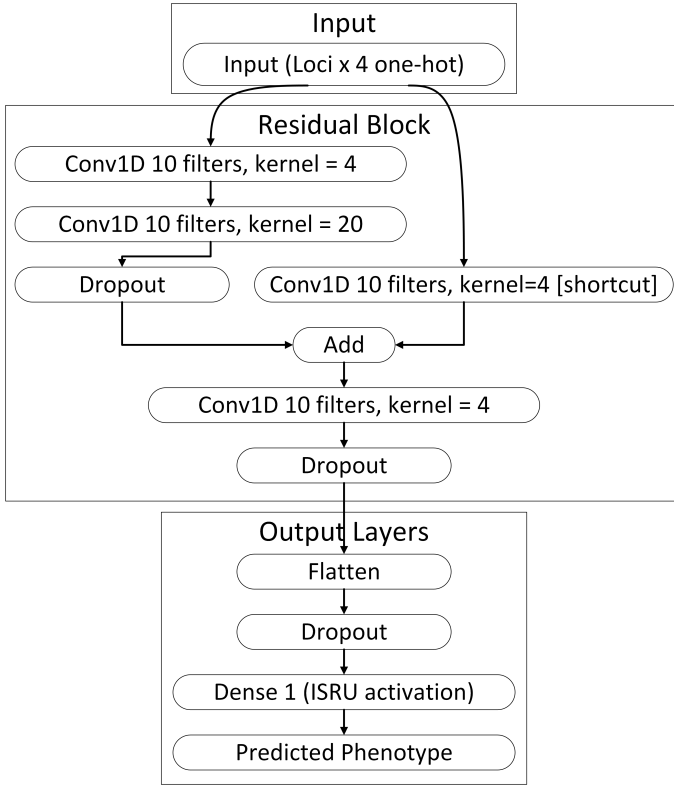


Fig. 1. Schematic of the 1D ResNet architecture for SNP-based phenotype prediction. The input is a one-hot encoded SNP sequence (dimensions: Loci $\times$ 4). A Residual Block applies two successive Conv1D layers (10 filters each) with different kernel sizes (4 and 20), followed by a dropout layer. A parallel shortcut Conv1D layer (kernel size 4) directly processes the input. The output of the residual block is the elementwise sum of the shortcut path and the main path (Add). This is passed through a third Conv1D and dropout. Finally, the Output Layers flatten the features and apply a fully-connected layer with ISRU activation to predict the phenotype.

In parallel, the network includes a shortcut path that implements the residual connection. The shortcut is a Conv1D with 10 filters of kernel size = 4 applied directly to the input SNP matrix (same as Conv1D Layer 1, but on the original input and without the subsequent conv). This layer produces its own $N \times 10$ feature map from the raw input. We likewise use linear activation for the shortcut conv.

Next, the output of the shortcut conv and the dropout output of the main conv stream are combined with a layer that performs element-wise addition (layers.add), yielding a summed feature map of shape $N \times 10$. This addition is the core of the ResNet residual block, equivalent to adding the identity (after a linear transform) to the output of the deeper conv path [5]. By doing so, the network can learn an easier residual mapping and mitigate vanishing gradients, as gradients can flow directly through the shortcut from later layers to earlier ones.

After the add operation, we include one more Conv1D Layer 3 with 10 filters, kernel size = 4 (same settings as before). This further processes the combined features, integrating the information from the residual merge. Another Dropout (same

default as before, 75%) is applied after this conv. At this point, we have a final set of convolutional features of shape $N \times 10$ (where N is the number of SNPs, and 10 features per SNP position).

We then add the output layers to interpret these features for prediction. We use a Flatten layer to collapse the spatial dimension, resulting in a 1D vector of length $N * 10$ for each sample. This is followed by a final Dropout (same default as before, 75%) to regularize the fully-connected layer. Finally, we have a Dense output layer with 1 unit that produces the predicted height. This dense layer uses a custom activation function called ISRU (Inverse Square Root Unit). The ISRU activation is defined as:

$$ISRU(x) = x / \sqrt{1 + \alpha(x^2)} \quad (1)$$

with a parameter α treated as a hyperparameter tuned for the scale of the target.

ISRU is a smooth, bounded activation function, as $x \rightarrow \pm\infty$, $ISRU(x) \rightarrow \pm \frac{1}{\sqrt{\alpha}}$, so it does not saturate to a hard limit but grows sub-linearly. It is smoother than ReLU or even tanh near 0, and it has been shown to prevent exploding gradients by keeping outputs in a controlled range. Carlile et al. note that ISRU (originally proposed as a variant of ISRLU) can speed up learning and improve generalization in deep networks by maintaining nonlinearity without harsh saturation [10]. In our context, using ISRU for the output layer helps in a regression setting: it gently scales large outputs and makes the network more robust to outliers in phenotype values, while still allowing a roughly linear relationship for small values (since $ISRU(x) \approx x$ for small x). The dense layer is also L2-regularized (especially on the bias term) to avoid bias shifts.

In summary, the model architecture combines a ResNet block (Conv $\rightarrow$ Conv $\rightarrow$ Dropout with a shortcut add) to capture multi-scale SNP interactions, and a custom activation to keep the regression stable. All convolution layers and the output layer include weight decay (L2 regularization coefficient 0.1 for conv weights, 0.01 for conv biases and dense bias) to discourage any single weight or SNP from dominating.

C. Training Procedure and Cross-Validation

We train the model using a 10-fold cross-validation strategy to maximize the usage of the limited data and to evaluate generalization. The fold number can also be changed to other integer numbers between 3 and 10. The validation and testing each take one fold, and the rest are for training.

For the default 10-folds, the data was split into 10 groups indexed 1 through 10 (provided in the input file). For each fold i (1 to 10) in turn, we performed the following assignment:

- **Test set:** fold i (all samples with fold index i).
- **Validation set:** fold $i + 1$ (or fold 1 if $i = 10$, wrapping around circularly).
- **Training set:** all other folds (the remaining 8 folds not in test or validation).

This scheme ensures that for each fold iteration, we train on 80% of the data, tune on 10%, and test on the remaining 10%, with no overlap. The “next” fold is taken as validation to make sure every fold gets to be tested once and validated once. The process results in 10 trained models (one for each test fold). We applied this cross-validation separately for the imputed and non-imputed datasets. In other words, for each fold i , we train two models with identical architecture: one on imputed SNP inputs and one on the non-imputed SNP inputs. This allows us to compare performance between using imputed versus raw genotype data directly.

We implemented training in TensorFlow Keras. To feed data efficiently, we created a custom data generator (SNPGenerator) that yields batches of one-hot encoded SNP data on the fly. The generator takes the int8 SNP matrix (for training or validation) and the corresponding target vector, and in its `__getitem__`, it slices a batch of SNP indices, then calls our one-hot conversion function on that slice to produce a batch of shape (batch_size, N, 4) which is then fed into the model. We set a relatively small batch size of 4. This small batch size was chosen to reduce memory usage per gradient step (each sample’s SNP one-hot matrix is large) and also empirically was enough to still get stable gradient estimates. Despite the small batch, we accumulate many weight updates over an epoch (since the training set might be thousands of samples). Training was done for up to 1000 epochs per fold, but we employed early stopping: we monitored the validation mean absolute error (MAE) and if it did not improve for 10 consecutive epochs, we stopped training early.

The loss function for training was mean squared error (MSE), since we are dealing with a regression problem (phenotype). However, our primary evaluation metric was the Pearson correlation coefficient (PCC) between predicted and actual phenotype value, which we computed after training.

D. Memory Optimization

One challenge in training was the memory footprint of the model, given the high-dimensional input. Our improvements in memory management were crucial: by using a generator and int8 storage, we avoided creating a giant in-memory one-hot array for all training data. We also took advantage of Keras functionality to clear the session after each fold training and invocation of the garbage collector to free up GPU/CPU memory. This way, each fold’s model could be trained fresh without lingering allocations from previous folds. Without these steps, we observed memory usage creeping up fold after fold (potentially exceeding 1 GB), whereas with the clean-up and on-the-fly generation, each fold run stayed around a few hundred MB at peak.

For each fold model, we saved the trained model weights to disk (in HDF5 .h5 format) so that we could later reload them for analysis. We also recorded the test set predictions to compute PCC. The pipeline was set up such that running all folds in succession would produce a log of each fold’s performance as well as final summary statistics.

TABLE I
PHENOTYPE DATA

Name	Type of Data	Number of SNPs	Short Description
Height	Numeric	4236	Plant height
Moisture	Numeric	4236	Moisture content
Oil	Numeric	4236	Oil content
Protein	Numeric	4236	Protein content

E. Saliency Mapping for SNP Importance

To make the model’s predictions interpretable, we performed saliency analysis on the trained models. Saliency mapping is a gradient-based technique commonly used in CNNs to highlight which input features most strongly influence the output. In image classification, saliency maps can highlight which pixels contribute to a class score. Here, we adapt it to our regression on SNP data: we want to know, for a given sample (or overall), which SNPs, when changed, would most affect the predicted phenotype.

After training all folds, we computed saliency values for each SNP across the test sets of all folds. To get a robust importance estimate for each SNP, we collected saliencies for every test sample in every fold and then took the average saliency for each SNP over all those samples. This yielded a single saliency score per SNP, averaged across individuals and cross-validation folds (so that the importance is not biased by a particular train/test split). Let $S_i^{(j)}$ be the saliency for SNP i in sample j ’s input, then we compute $\bar{S}_i = \frac{1}{M} \sum_j S_i^{(j)}$ where M is the total number of test predictions across all folds. Because each fold’s test set is disjoint, M equals the total sample count.

We then visualize these average saliencies in a saliency plot. In addition, we identify the top SNPs by saliency by ranking $\bar{S}_i$ values. The top K SNP names and their saliency scores are saved for downstream analysis (default $K = 20$).

III. EXPERIMENTS AND RESULTS

A. Experimental Data

A dataset of plant phenotypes (height, moisture, oil, and protein) with the corresponding SNP genotypes summarized in Table I. Each data instance consists of a continuous phenotype value (the target) and a vector of SNP values spanning the genome.

The SNPs are bi-allelic and encoded numerically as 0, 1, or 2 (representing, e.g., the count of reference alleles). In particular, each SNP is converted into a 4-dimensional binary vector: for genotype values 0, 1, 2 we assign distinct one-hot vectors, and we reserve a fourth category for missing data as seen in Table II.

B. Parameters

Most of our parameters are the same as the original setup by Liu et al [5]. Our main changes are decreasing the batch size to 4 and increasing the early stopping patience to 10.

We set $\alpha = 0.03$ for our height prediction task (this was treated as a hyperparameter tuned for the scale of the target).

TABLE II
ONE-HOT ENCODING CONVERSION (WHERE A = REFERENCE ALLELE
AND A = ALTERNATE ALLELE).

Name	Array Conversion	Number Representation
Homozygous reference genotype (AA)	[0,0,1,0]	0
Heterozygous genotype (Aa)	[0,1,0,0]	1
Homozygous alternate (aa)	[0,0,0,1]	2
Missing genotype	[1,0,0,0]	-1

We identify the top SNPs by saliency by ranking $\bar{S}_i$ values. The top K SNP names and their saliency scores are saved for downstream analysis (default $K = 20$).

C. Prediction Accuracy

We evaluated the model’s performance in terms of the Pearson correlation coefficient (PCC) between predicted and actual phenotypes. As shown in Table III, after 10-fold cross-validation for the height phenotype, the average PCC on the imputed dataset was approximately 0.46, while on the non-imputed dataset it was about 0.61 for the height. For the other phenotypes, the results are similar to the original paper by Liu et al [5]. For moisture, we had imputed and non-imputed values of 0.41 and 0.45, respectively. For oil, the results were similar, with 0.41 for imputed and 0.64 for non-imputed. Lastly, protein had an imputed PCC with 0.39 but a non-imputed PCC of 0.59.

The results for each fold were fairly consistent, with non-imputed data yielding a higher correlation in almost every fold. This reflects that every fold scores higher on the non-imputed data. One plausible explanation is that the imputation procedure introduced systematic noise or biased allele frequencies, obscuring genuine genotype–phenotype signals. Because our network learns directly from the categorical one-hot patterns, mis-imputed SNPs can act as misleading features, whereas explicit “missing” indicators in the raw data may be easier for the model to treat as neutral. These findings suggest that, for this dataset, leaving missing values explicit and allowing the network to learn a “missing” pattern is preferable to statistical imputation, a result worth investigating in future work with alternative imputation schemes or masking strategies.

In practical terms, a correlation of 0.45 for height means the model explains around 20% of the variance (since $R^2 = 0.20$), which is not a respectable outcome for a complex trait influenced possibly by the environment. The non-imputed performance (PCC 0.61, $R^2 \approx 0.37$) is slightly lower, likely reflecting noise introduced by missing data or the model’s difficulty in learning a “missing genotype” pattern. The results also align with those reported by Liu et al. [5] for soybean: their CNN achieved correlation improvements over baseline for traits, though in classification tasks (Chen et al. [6]) accuracy gains were more dramatic.

It is worth noting that the training losses (MSE) continuously decreased, and validation MAE stabilized typically around 8–10 (in the units of height). We prioritized correlation as the metric since it is scale-invariant. For example, if the

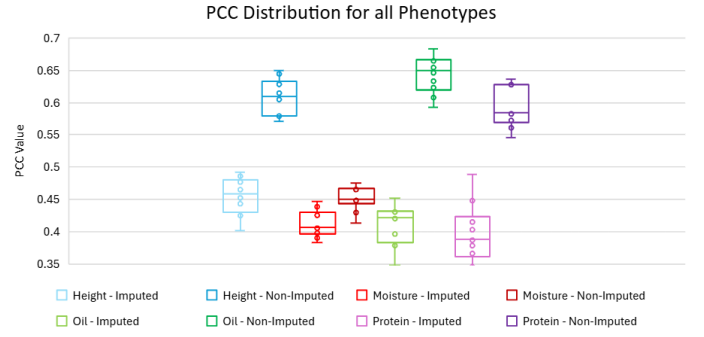


Fig. 2. All PCC values for each phenotype across all 10 folds during our second run.

TABLE III
COMPARISON OF CNN PCC RESULTS ACROSS IMPUTED AND
NON-IMPUTED PHENOTYPES

	Original Paper (dualCNN)	Our Results (Average Across 5 Runs)
Height (imp/non-imp)	0.465 / 0.615	0.459 / 0.61
Moisture (imp/non-imp)	0.426 / 0.463	0.41 / 0.45
Oil (imp/non-imp)	0.412 / 0.668	0.408 / 0.638
Protein (imp/non-imp)	0.402 / 0.619	0.392 / 0.589

model predicts height values that are off by a constant factor, MSE would be high, but correlation could still be 1. In our case, the model did not show significant bias issues; predictions were roughly on the correct scale due to the ISRU preventing extreme outputs. However, high PCC indicates the model ranks individuals correctly by the phenotype to a good extent, which is often the goal in breeding (identifying the tallest, etc.). No overfitting was observed: the validation and test metrics were similar, aided by early stopping as shown in Figure 2.

D. Saliency Analysis and Important SNPs

A key outcome of our approach is the interpretability gained via saliency maps. We computed the average saliency for each SNP as described above, and plotted the values across all SNPs (around 4,200) in Figure 3. The saliency plot reveals a sparse pattern: most SNPs have very low saliency (near 0), but a handful of SNPs stand out with much higher values. These top SNPs by saliency are likely the most influential genetic markers for height according to our model. Other phenotypes show similar trends as well.

From Figure 3, we see that the highest saliency values reach around 0.02 – 0.023. To concretely identify important loci, Table IV and Table V lists the top five SNPs ranked by saliency score:

Comparing our top SNPs to the original paper, all of them matched. This included Gm12_2659260 for our height phenotype, in which the region around it is “a copy number variation signal that is associated with plant height” [5]. Our most prominent SNP for moisture was Gm17_6781998, and its closest gene Glyma17g09165 is related to “cold, wounding, salt stress, and mannitol stimulus” [5]. For oil, we identified

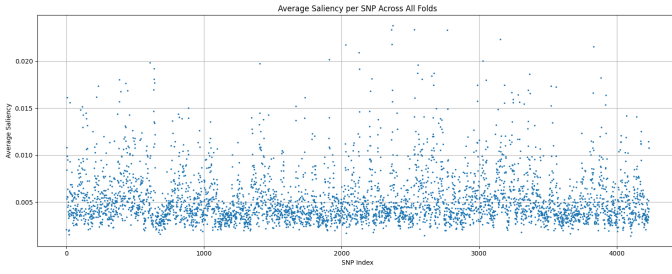


Fig. 3. Average saliency value for each SNP across all cross-validation folds for the height phenotype using our second run. Each blue dot represents one SNP's saliency (averaged over all test samples). The majority of SNPs have low saliency (below 0.005), while a small subset of SNPs have significantly higher saliency values, indicating greater influence on the height prediction.

TABLE IV

TOP 5 HEIGHT AND MOISTURE SNPs BY AVERAGE SALIENCY IN RUN 2. ("GM" REFERS TO GLYCINE MAX CHROMOSOME IDENTIFIERS AS PER THE DATASET.)

Height		Moisture	
SNP	Saliency	SNP	Saliency
Gm12_2894203	0.023934	Gm17_6781998	0.022769
Gm12_2659260	0.023506	Gm15_9530676	0.018826
Gm13_2550479	0.023498	Gm18_8451185	0.018535
Gm14_398773	0.023455	Gm07_40760824	0.017965
Gm15_48737423	0.022476	Gm18_12926582	0.017585

Gm15_48737423 in which its gene region (Glyma15g41600) "involves a sulfur amino acid metabolic process, a cysteine biosynthetic process, and a cell wall modification" [5]. Lastly, for protein, we identified prominent SNPs on chromosomes 2 and 20, which may be associated with protein content [5]. The presence of multiple top signals on the same chromosome could imply a cluster of influential markers or a linkage disequilibrium block that the model is sensitive to. Other top saliency SNPs came from different chromosomes, implying that the model is picking up polygenic signals in multiple genomic regions, not just one.

One interesting feature was the interactive saliency viewer we built into the console: after running the summary, one can input an SNP name and get its average saliency reported. For example, querying "Gm13_29736067" for height returned an average saliency of 0.018551, whereas a random low-saliency SNP might return something like 0.000500. This allows users to quickly check whether specific SNPs (perhaps ones previously reported in literature) were picked up by

TABLE V

TOP 5 OIL AND PROTEIN SNPs BY AVERAGE SALIENCY. ("GM" REFERS TO GLYCINE MAX CHROMOSOME IDENTIFIERS AS PER THE DATASET.)

Oil		Protein	
SNP	Saliency	SNP	Saliency
Gm02_5591380	0.022087	Gm07_7832406	0.019311
Gm15_48737423	0.020722	Gm18_20807357	0.018281
Gm04_8184443	0.020188	Gm20_29976653	0.017121
Gm06_7558353	0.019364	Gm14_4561500	0.016828
Gm12_2762306	0.01723	Gm17_10458588	0.016523

the model. In practice, a breeder or geneticist could use the saliency output to guide further study: SNPs with high saliency could be investigated for nearby genes or used in marker-assisted selection.

IV. DISCUSSIONS AND CONCLUSIONS

This work demonstrates that modern deep learning models can be made interpretable and useful for GWAS-style analysis. The combination of CNN and saliency analysis can be thought of as an automated, non-linear GWAS. The experimental results demonstrate that our saliency-aware ResNet can achieve mediocre predictive accuracy for a few of our phenotypes (quantitative traits), using only SNP features. For height, a PCC of 0.61 (non-imputed data) means the model captures a substantial portion of the genetic signal. The saliency analysis adds confidence to our model's validity. The fact that known or expected important SNPs (as per GWAS) emerged with high saliency indicates that the network likely learned genuine genotype-phenotype relationships rather than overfitting to the noise.

The pipelines developed in this study can complement traditional GWAS by capturing interactions and using all SNPs simultaneously rather than testing one by one. Importantly, the entire workflow is encapsulated in a portable software container, ensuring that others can reproduce and build upon our results. We believe that this approach can be applied to other complex traits and will be valuable in further research contexts, where understanding the genetic basis of a trait is as important as predicting it.

REFERENCES

- [1] E. Uffelmann, Q. Q. Huang, N. S. Munung, J. de Vries, Y. Okada, A. R. Martin, H. C. Martin, T. Lappalainen, and D. Posthuma, "Genome-wide association studies," *Nature Reviews Methods Primers*, vol. 1, p. 59, 2021. [Online]. Available: <https://doi.org/10.1038/s43586-021-00056-9>
- [2] R. Varshney, M. Pandey, A. Bohra, and N. Singh, "Genome-wide association study of agronomic traits in pearl millet using bayesian models," *Plant Genome*, 2021.
- [3] N. Puthiyedth, F. Zeinalinesaz, D. Hou, Y. Zhang, W. Lin, and Y. Yan, "Leveraging lasso-based methodologies for enhanced snp analysis in plant genomes," *Bioinformatics Advances*, vol. 5, no. 1, p. vbaf014, 2025.
- [4] Y. Yan, C. Burbridge, J. Shi, J. Liu, and A. Kusalik, "Effects of input data quantity on genome-wide association studies (gwas)," *International Journal of Data Mining and Bioinformatics*, vol. 22, no. 1, pp. 19–43, 2019.
- [5] Y. Liu, D. Wang, F. He, J. Wang, T. Joshi, and D. Xu, "Phenotype prediction and genome-wide association study using deep convolutional neural network of soybean," *Frontiers in genetics*, vol. 10, p. 1091, 2019.
- [6] X. Chen, D. G. Chen, Z. Zhao, J. Zhan, C. Ji, and J. Chen, "Artificial image objects for classification of schizophrenia with gwas-selected snvs and convolutional neural network," *Patterns*, vol. 2, no. 8, 2021.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [8] Y. Li, C. J. Willer, J. Ding, P. Scheet, and G. R. Abecasis, "Mach: using sequence and genotype data to estimate haplotypes and unobserved genotypes," *Genetic epidemiology*, vol. 34, no. 8, pp. 816–834, 2010.
- [9] A. Xavier, W. Muir, and K. Rainey, "Assessing predictive properties of genome-wide selection in soybeans. g3 genes—genomes—genetics, 6, 2611–2616," 2016.
- [10] B. Carlile, G. Delamarter, P. Kinney, A. Marti, and B. Whitney, "Improving deep learning by inverse square root linear units (isrlus)," *arXiv preprint arXiv:1710.09967*, 2017.